

Tarea 2

Eckert Gibreel Baeza Gómez

24 de Agosto 2026

1 Modelos de Datos

1.1 Vectorial

Representa objetos geográficos mediante coordenadas x,y, utilizando puntos, líneas y polígonos, y los relaciona con información descriptiva sobre esos objetos.

1.1.1 Características

- **Representación numérica densa (embeddings):** Los datos no estructurados (texto, imágenes, audio, vídeo) se transforman en listas de números reales (vectores de alta dimensionalidad, típicamente entre 128 y 4096 dimensiones) mediante modelos de aprendizaje automático (por ejemplo, Word2Vec, BERT, ResNet, CLIP). Cada dimensión codifica una característica latente del contenido.
- **Generación automática a partir de datos no estructurados:** A diferencia del modelo geoespacial, los vectores no se “dibujan” manualmente mediante coordenadas, sino que se *extraen* automáticamente mediante modelos de lenguaje o visión por computador. Estos modelos capturan relaciones contextuales complejas y semántica profunda.
- **Almacenamiento en motores especializados o convergentes:** Los vectores se gestionan mediante:
 - Bases de datos vectoriales nativas (Milvus, Qdrant, Pinecone, Weaviate, Chroma).
 - Extensiones sobre motores relacionales tradicionales (pgvector en PostgreSQL, AI Vector Search en Oracle Database 23ai).
 - Motores de búsqueda con soporte vectorial (Elasticsearch, OpenSearch, Vespa).
- **Indexación específica para búsqueda por vecinos más cercanos (k-NN):** Para evitar explorar todo el espacio vectorial (lo que sería computacionalmente inviable), se utilizan índices aproximados como:
 - **HNSW** (Hierarchical Navigable Small World): Grafo jerárquico que permite búsquedas rápidas en alta dimensionalidad.
 - **IVF** (Inverted File Index): Partición del espacio vectorial en clusters para reducir el área de búsqueda.
 - **Cuántificación escalar o producto:** Técnicas de compresión que reducen el tamaño de los vectores y aceleran las distancias.

Estos índices permiten consultas de similitud en milisegundos sobre miles de millones de vectores.

- **Consultas por similitud semántica:** La posición del vector en el espacio multidimensional codifica el significado. Dos elementos con contenido similar (ej. “perro” y “can”) tienen vectores cercanos. Las consultas se resuelven calculando la distancia o el ángulo entre vectores, usando métricas como:
 - **Distancia euclídea:** medida directa de la separación lineal.
 - **Similitud del coseno:** ángulo entre vectores, muy utilizada para comparar orientación semántica.
 - **Producto escalar:** combinación de magnitud y dirección.

- **Integración con datos estructurados (consulta híbrida):** Permite combinar filtros exactos (SQL) con búsquedas semánticas en una misma operación. Por ejemplo, “muéstrame los 10 documentos más similares al texto X, pero solo los publicados en 2025 y de la categoría ‘finanzas’”. Esta capacidad es nativa en sistemas convergentes como Oracle 23ai o mediante extensiones como pgvector.
- **Alta dimensionalidad y requisitos computacionales:** Manejar vectores de cientos o miles de dimensiones implica operaciones lineales (productos escalares, proyecciones) que requieren optimización a nivel de CPU y, en muchos casos, aceleración por GPU (especialmente en índices HNSW y en el cálculo masivo de distancias).
- **Independencia de estructura fija:** A diferencia de las bases de datos relacionales, el modelo vectorial no requiere un esquema predefinido. Los vectores se almacenan como un tipo de dato adicional junto con metadatos (payload) que pueden ser arbitrarios y flexibles (JSON, etiquetas, marcas temporales).
- **Objetos individuales con identidad y actualización granular:** Cada vector es una entidad independiente con un identificador único. Esto permite añadir, modificar o eliminar vectores sin afectar al resto de la base de datos, facilitando el mantenimiento de colecciones dinámicas (ej. actualizar embeddings de nuevos documentos sin reindexar todo el conjunto).
- **Escalabilidad horizontal distribuida:** Los sistemas vectoriales modernos (Milvus, Qdrant, Elasticsearch) están diseñados para escalar añadiendo más nodos, distribuyendo tanto el almacenamiento como la carga de consultas, lo que permite manejar volúmenes de miles de millones de vectores con tiempos de respuesta predecibles.
- **Soporte para diferentes tipos de datos de origen:** El modelo vectorial es agnóstico al tipo de dato original. Puede representar:
 - Texto (mediante modelos como sentence-transformers o LLM embeddings).
 - Imágenes (mediante redes convolucionales como ResNet o ViT).
 - Audio (mediante modelos como wav2vec).
 - Vídeo (mediante fusión de embeddings de frames y audio).
 - Datos tabulares (mediante embeddings de redes neuronales para características numéricas y categóricas).

1.1.2 Ventajas

- **Alta precisión y exactitud:** Permite representar con gran detalle la ubicación y forma de los objetos mediante coordenadas (X, Y). Por ejemplo, un punto puede representar un árbol, una línea una carretera y un polígono un terreno, manteniendo la fidelidad al mundo real sin la pérdida de información que ocurre en el modelo ráster.
- **Representación eficiente de objetos discretos:** Es muy adecuado para elementos que tienen límites definidos y fronteras claras, como edificios, carreteras, municipios, ríos o parcelas. Almacenar estos elementos mediante vértices suele requerir mucho menos espacio de almacenamiento que hacerlo con celdas (ráster).
- **Almacenamiento enriquecido de atributos:** A cada objeto geométrico (punto, línea o polígono) se le pueden asociar datos descriptivos en una tabla independiente (ej. nombre, población, superficie, estado al que pertenece, o incluso metadatos complejos). Estos atributos se vinculan mediante un identificador único, facilitando la gestión de la información.
- **Consultas espaciales y topológicas avanzadas:** Permite realizar preguntas complejas relacionadas con la posición y relaciones entre objetos. Por ejemplo:
 - ¿Qué edificios están dentro de este municipio? (Contención)
 - ¿Qué hospitales están a menos de 5 km de un punto? (Proximidad)
 - ¿Qué carreteras intersectan esta zona? (Intersección)
 - ¿Qué municipios comparten una frontera? (Adyacencia y conectividad)

Gracias a la topología (conectividad, definición de área y contigüidad), el sistema entiende las relaciones vecinales sin necesidad de duplicar geometrías.

- Alto rendimiento y velocidad en consultas (ventaja de base de datos): Las bases de datos vectoriales modernas (como las implementadas en RDBMS) utilizan estructuras de indexación especializadas (como R-trees, k-d trees o índices de similitud) y técnicas de procesamiento paralelo. Esto permite realizar búsquedas espaciales y de similitud sobre conjuntos de datos masivos (millones de registros) en milisegundos, superando ampliamente a las bases de datos relacionales tradicionales sin optimizar.
- Escalabilidad horizontal y manejo de grandes volúmenes: Gracias a técnicas como el sharding, el particionado y el clustering (ej. Oracle RAC), estas bases de datos pueden crecer con el negocio. Manejan desde petabytes de datos geoespaciales hasta enormes colecciones de embeddings sin degradar el rendimiento, lo que las hace ideales para entornos empresariales y de IoT.
- Versatilidad para datos complejos y no estructurados (IA y embeddings): Además de la geometría tradicional, el modelo vectorial (en el sentido de embeddings matemáticos) permite almacenar y buscar el significado semántico de datos no estructurados como textos, imágenes, audio o vídeo. Esto habilita casos de uso avanzados como:
 - Motores de recomendación y búsqueda semántica.
 - Detección de fraudes en tiempo real (analizando patrones transaccionales).
 - Sistemas de recuperación aumentada (RAG) para potenciar modelos de lenguaje (LLMs) con datos propios de la empresa.
 - Análisis de sentimiento y clasificación de documentos.
- Integración sencilla con el ecosistema de datos: Se integra fácilmente con pipelines ETL, herramientas de analítica (mediante extensiones SQL), y sistemas de terceros a través de APIs estandarizadas. Al ser parte de una base de datos convergente (como Oracle 23ai), elimina la necesidad de mover o sincronizar datos entre sistemas separados (evitando el *data sprawl*), lo que garantiza consistencia y reduce la complejidad operativa.
- Edición y mantenimiento eficiente: Permite modificar la geometría o los atributos de un objeto individual (ej. dividir una parcela, actualizar el nombre de una calle) sin tener que reescribir o regenerar toda la base de datos, lo que facilita la actualización constante de la información.

En esencia, la gran ventaja del modelo vectorial (integrado en una base de datos moderna) es que representa objetos del mundo real de manera precisa y estructurada, permitiendo relacionarlos con información descriptiva y, además, ejecutar análisis espaciales, semánticos y de IA a gran velocidad y escala, todo dentro de una plataforma unificada y segura.

1.1.3 Desventajas

- Mayor complejidad estructural: Su estructura (nodos, arcos, vértices y polígonos) puede ser más compleja que la de un modelo ráster, especialmente cuando se manejan muchas relaciones topológicas y capas de información interconectadas. Esta complejidad se traslada a la administración de la base de datos.
- No es ideal para fenómenos continuos o graduales: Es menos adecuado para representar fenómenos que cambian suavemente en el espacio sin límites abruptos, como temperatura, elevación, precipitación o niveles de contaminación. Para estos casos, el modelo ráster suele ser más natural y eficiente.
- Alto coste computacional en operaciones complejas (overlay): Aunque los índices mejoran el rendimiento, las operaciones espaciales complejas (como intersecciones, uniones, diferencias simétricas o disoluciones entre múltiples polígonos) son computacionalmente costosas. Si los datasets son muy grandes, estos procesos requieren una cuidadosa optimización y pueden consumir muchos recursos de CPU y memoria.
- Propensión a errores topológicos si los datos están mal contruidos: Pueden aparecer errores como polígonos superpuestos, espacios vacíos (slivers) entre polígonos adyacentes, líneas que no llegan a conectarse (*undershoot*) o que se pasan del punto de unión (*overshoot*). Para corregirlos, se necesitan herramientas de validación topológica y ajustes de tolerancia al agrupamiento (*cluster tolerance*), lo que añade trabajo de depuración.

- Mayor dificultad para representar variaciones internas: A diferencia del ráster, donde cada celda puede tener un valor, en el modelo vectorial representar variaciones dentro de un mismo polígono (ej. diferentes tipos de suelo dentro de una misma parcela) requiere dividir el polígono en partes más pequeñas, aumentando la complejidad de los datos.
- Coste y esfuerzo en la adquisición y creación de datos: Obtener geometrías vectoriales precisas puede requerir levantamientos topográficos costosos, digitalización manual de mapas, procesamiento de imágenes satelitales o técnicas avanzadas de fotogrametría, lo que implica una inversión inicial significativa.
- Requisitos de infraestructura para IA y grandes volúmenes (nuevo): Si se utilizan para embeddings de IA o aprendizaje automático, su implementación plena exige infraestructura de hardware especializada (como GPUs de alto rendimiento), así como un manejo cuidadoso de la semántica y las versiones de los modelos, lo que añade una capa adicional de complejidad técnica que no se da en el modelo ráster tradicional.

En esencia, la principal desventaja del modelo vectorial en bases de datos es que su complejidad estructural y computacional se incrementa notablemente con el volumen de datos y la complejidad de las relaciones, requiriendo hardware potente, datos de entrada muy limpios y un conocimiento técnico especializado para explotar todo su potencial sin caer en errores de rendimiento o precisión.

1.1.4 ¿Cuándo utilizarlo?

Se debe utilizar cuando el objetivo es buscar, recomendar o analizar información basándose en su similitud conceptual, más allá de coincidencias textuales exactas.

Casos de uso ideales

- **Búsqueda semántica y recuperación de información:** en colecciones masivas de documentos, correos electrónicos, artículos o bases de conocimiento, donde el usuario realiza preguntas en lenguaje natural y el sistema devuelve los resultados más relevantes por cercanía semántica.
- **Sistemas de recomendación:** para comparar perfiles de usuarios con características de productos (películas, libros, artículos, servicios) y sugerir aquellos que resultan más afines.
- **Clasificación y agrupamiento automático (clustering):** para organizar grandes volúmenes de texto en categorías temáticas o detectar grupos de clientes con comportamientos similares sin intervención manual.
- **Modelos de lenguaje aumentados (RAG - Retrieval-Augmented Generation):** para alimentar a un modelo de lenguaje (LLM) con información propia y actualizada de la empresa, recuperando fragmentos relevantes desde la base de datos vectorial antes de generar cada respuesta.
- **Detección de anomalías y fraudes:** para analizar en tiempo real patrones transaccionales representados como vectores, identificando comportamientos atípicos o sospechosos.
- **Búsqueda multimedia:** para encontrar imágenes, audios o vídeos visual o auditivamente similares a partir de embeddings generados por redes neuronales.
- **Análisis de sentimiento y clasificación de textos:** cuando se necesita categorizar opiniones, reseñas o comentarios según su tono (positivo, negativo, neutral) utilizando representaciones vectoriales densas.

¿Cuándo NO es recomendable? El modelo vectorial en bases de datos no es adecuado en los siguientes escenarios:

- Cuando se requieren búsquedas **exactas y puramente léxicas**, como la localización de un código de producto, un número de identificación o una palabra clave sin necesidad de sinónimos o contexto.
- Cuando los datos ya están **perfectamente estructurados** en tablas relacionales y las consultas son exclusivamente de tipo SELECT, JOIN o agregación numérica, sin necesidad de similitud semántica.

- Cuando el volumen de datos es muy reducido y el coste computacional de generar y mantener embeddings (vectorización) no se justifica frente al uso de motores de búsqueda textual tradicionales (como índices invertidos).

En estos casos, una base de datos relacional tradicional o un motor de búsqueda booleano resultan más simples, rápidos y eficientes.

1.1.5 Software que lo implementa

Existe un ecosistema amplio y diverso de sistemas que implementan el modelo vectorial en el contexto de bases de datos. Estos pueden clasificarse en varias categorías según su modelo de despliegue, licenciamiento y caso de uso principal.

Bases de datos vectoriales especializadas (propósito específico) Son sistemas diseñados desde cero para almacenar y consultar vectores de alta dimensionalidad, ofreciendo un rendimiento optimizado para búsquedas de similitud.

- **Pinecone:** Base de datos vectorial completamente gestionada (*serverless*) y de carácter comercial. Está diseñada para entornos empresariales que requieren escalabilidad, alta disponibilidad y cero operaciones de infraestructura. Es una opción popular para aplicaciones de IA en producción.
- **Milvus** (y su versión gestionada Zilliz): Sistema de código abierto (Apache-2.0) optimizado para manejar miles de millones de vectores en entornos distribuidos. Ofrece soporte para aceleración por GPU, múltiples tipos de índices (HNSW, IVF, DiskANN) y una arquitectura desacoplada que permite escalar de forma independiente cada componente.
- **Qdrant:** Base de datos vectorial de código abierto escrita en Rust, conocida por su alto rendimiento y su excelente capacidad de filtrado sobre metadatos (*payload filtering*). Soporta despliegue auto-gestionado o en la nube, y es especialmente adecuada para cargas de trabajo de hasta decenas de millones de vectores con filtrado complejo.
- **Weaviate:** Base de datos vectorial de código abierto con un enfoque en la nube nativa y la simplicidad de uso. Ofrece una API GraphQL, soporte nativo para búsqueda híbrida (vectorial + textual) y capacidad para generar incrustaciones (*embeddings*) directamente desde la base de datos.
- **Chroma:** Base de datos vectorial de código abierto, ligera y enfocada en la facilidad de uso para prototipado rápido y entornos de desarrollo local. Es una opción popular para pruebas de concepto y proyectos de pequeña escala.
- **LanceDB:** Base de datos vectorial de código abierto diseñada para aplicaciones *embedded* y multimodales, con un enfoque en la eficiencia de almacenamiento y la integración con el ecosistema de datos moderno.

Extensiones vectoriales sobre bases de datos existentes Permiten añadir capacidades vectoriales a motores de base de datos ya consolidados, evitando la introducción de un sistema adicional y aprovechando la infraestructura y los conocimientos existentes.

- **pgvector:** Extensión de código abierto para PostgreSQL que añade el tipo de dato vectorial y soporte para índices HNSW e IVFFlat. Es la opción recomendada para equipos que ya utilizan PostgreSQL y desean incorporar búsqueda semántica sin añadir complejidad operativa. Soporta filtrado mediante SQL estándar y se integra con el ecosistema de Postgres.
- **Oracle AI Vector Search:** Capacidad nativa de Oracle Database 23ai que incorpora el tipo de dato **VECTOR**, índices vectoriales y operadores SQL para búsqueda por similitud. Permite almacenar incrustaciones junto con los datos transaccionales de la empresa y realizar consultas híbridas que combinan condiciones de negocio con búsqueda semántica en una misma sentencia SQL.

- **MongoDB Atlas Vector Search:** Funcionalidad integrada en la base de datos documental MongoDB que permite realizar búsquedas vectoriales y combinarlas con filtros JSON y búsqueda de texto completo. Es una opción atractiva para equipos que ya utilizan MongoDB como base de datos principal.
- **Redis Stack:** Extensión del conocido sistema en memoria Redis que añade capacidades de búsqueda vectorial con latencia ultrarrápida, ideal para aplicaciones que requieren respuestas en tiempo real.

Motores de búsqueda con capacidades vectoriales Plataformas originalmente diseñadas para búsqueda de texto que han incorporado soporte para vectores, ofreciendo búsqueda híbrida (combinando relevancia semántica y coincidencia léxica).

- **Elasticsearch y OpenSearch:** Motores de búsqueda distribuidos y maduros que soportan campos vectoriales y búsqueda por vecinos más cercanos (k-NN). Son especialmente potentes para casos de uso que requieren combinar búsqueda semántica con filtrado avanzado y búsqueda textual tradicional.
- **Vespa:** Plataforma de búsqueda y recomendación a gran escala desarrollada por Yahoo, que soporta búsqueda vectorial, búsqueda de texto y razonamiento sobre datos estructurados en un mismo sistema. Está optimizada para despliegues de escala billonaria.

Plataformas de datos convergentes Sistemas que integran capacidades vectoriales dentro de una arquitectura de base de datos más amplia, permitiendo gestionar vectores junto con otros tipos de datos y cargas de trabajo.

- **TiDB Vector Search:** Funcionalidad vectorial de TiDB, una base de datos distribuida de código abierto compatible con SQL. Ofrece despliegue gestionado o auto-gestionado y soporte para índices HNSW.
- **Infino:** Motor de búsqueda que ejecuta consultas SQL, búsqueda de texto completo y búsqueda vectorial sobre una única copia de los datos almacenada en formato Parquet sobre almacenamiento de objetos.

Cada una de estas herramientas responde a necesidades diferentes: desde el prototipado rápido con Chroma o pgvector, pasando por cargas de trabajo empresariales con Qdrant o Milvus, hasta la integración nativa con sistemas transaccionales mediante Oracle Database 23ai o PostgreSQL con pgvector. La elección del software adecuado depende del volumen de datos, los requisitos de filtrado, el modelo de despliegue deseado y la infraestructura existente en la organización.

1.2 Big Data

El modelo de datos *big data* no se refiere a una estructura de datos única y rígida (como el modelo relacional o el vectorial), sino a un **paradigma integral de gestión, almacenamiento y análisis** diseñado para manejar conjuntos de información que superan las capacidades de las herramientas tradicionales (bases de datos relacionales y hojas de cálculo). Este modelo abarca tanto la diversidad de formatos como la infraestructura tecnológica necesaria para extraer valor de datos masivos, heterogéneos y de rápida actualización.

Características fundamentales El modelo *big data* se define por cinco atributos esenciales que lo diferencian de los enfoques clásicos:

- **Volumen:** La cantidad de datos es inmensa, medida en terabytes o petabytes. Proviene de fuentes como transacciones digitales, sensores IoT, redes sociales y registros de aplicaciones.
- **Velocidad:** La rapidez con la que los datos son generados y deben ser procesados. Exige capacidades de *streaming* y procesamiento en tiempo real o casi real, en lugar de únicamente lotes programados.
- **Variedad:** Abarca datos *estructurados* (tablas SQL), *semiestructurados* (JSON, XML) y *no estructurados* (texto libre, imágenes, audio, vídeo). Esta diversidad obliga a abandonar los esquemas rígidos.
- **Veracidad:** La calidad, fiabilidad y precisión de los datos. Dado que provienen de fuentes muy diversas, requieren procesos de limpieza, validación y gobierno para evitar que errores o ruido distorsionen los análisis.
- **Valor:** El beneficio tangible que se obtiene al analizar los datos. Sin un análisis que transforme los datos en *insights* accionables, el resto de las V carece de propósito.

1.2.1 Ventajas

- **Mejora en la toma de decisiones:** Al disponer de conjuntos masivos y diversos, las organizaciones pueden basar sus decisiones en patrones, correlaciones y tendencias que antes permanecían ocultos. Esto permite pasar de estrategias intuitivas a estrategias basadas en evidencia cuantitativa.
- **Personalización de la experiencia del cliente:** El análisis de grandes volúmenes de datos de comportamiento, preferencias y transacciones permite crear perfiles detallados de clientes. Esto facilita ofrecer recomendaciones, promociones y servicios adaptados a cada individuo, aumentando la fidelización y el compromiso.
- **Optimización de la eficiencia operativa:** Los datos en tiempo real provenientes de sensores, registros de procesos y cadenas de suministro permiten identificar cuellos de botella, predecir fallos en equipos (mantenimiento predictivo) y ajustar la producción o la logística de forma dinámica, reduciendo costes y tiempos de inactividad.
- **Desarrollo de productos y servicios receptivos:** El análisis de la retroalimentación de los usuarios y de las métricas de uso permite a las empresas iterar rápidamente sobre sus productos, priorizando características que generan mayor valor y corrigiendo defectos de forma ágil.
- **Mejora en la gestión de riesgos y detección de fraudes:** En sectores como el financiero, el big data permite analizar patrones transaccionales en tiempo real para identificar comportamientos anómalos, reduciendo pérdidas por fraudes y cumpliendo con regulaciones más estrictas.
- **Innovación en sectores críticos:** En salud, por ejemplo, el análisis masivo de registros médicos, datos genómicos y dispositivos *wearable* posibilita diagnósticos más precisos, tratamientos personalizados y detección temprana de enfermedades.
- **Escalabilidad horizontal y flexibilidad:** Gracias a arquitecturas distribuidas (Hadoop, Spark) y almacenamiento en la nube, el modelo puede crecer añadiendo más nodos sin necesidad de rediseñar toda la infraestructura, adaptándose a aumentos impredecibles del volumen de datos.
- **Capacidad para integrar datos heterogéneos:** Al aceptar datos estructurados, semiestructurados y no estructurados, el modelo rompe los silos de información y permite obtener una visión unificada del negocio, enriqueciendo los análisis con fuentes antes inaccesibles.

1.2.2 Desventajas

- **Complejidad arquitectónica y técnica:** La implementación de un ecosistema *big data* requiere la integración de múltiples herramientas (Hadoop, Spark, Kafka, bases de datos NoSQL, sistemas de orquestación, etc.), lo que aumenta la complejidad de diseño, despliegue y mantenimiento.
- **Altos costes de infraestructura y almacenamiento:** Aunque el almacenamiento en la nube ha reducido costes, manejar petabytes de datos sigue siendo caro, especialmente cuando se requieren altas velocidades de procesamiento y redundancia para garantizar disponibilidad.
- **Calidad y gobierno de datos:** La procedencia de múltiples fuentes y la falta de esquemas rígidos introducen ruido, duplicidades e inconsistencias. Garantizar la veracidad de los datos exige invertir en procesos continuos de limpieza, validación y gobernanza, que pueden consumir entre el 50 % y el 80 % del tiempo de los científicos de datos.
- **Brecha de habilidades y talento:** La escasez de profesionales con conocimientos en ingeniería de datos, ciencia de datos y administración de plataformas distribuidas es uno de los mayores obstáculos. Las organizaciones a menudo deben invertir en formación o en contratación costosa.
- **Privacidad y seguridad:** Almacenar grandes volúmenes de información sensible (clientes, transacciones, salud) aumenta el riesgo de filtraciones y el impacto de posibles brechas. Además, el cumplimiento de regulaciones como el RGPD o la HIPAA añade capas de complejidad en el manejo de los datos.

- **Integración con sistemas heredados:** Muchas organizaciones aún dependen de bases de datos relacionales y procesos ETL tradicionales. Migrar o convivir con estas infraestructuras antiguas puede ser lento y costoso, y requiere de una estrategia de modernización escalonada.
- **Rápida evolución tecnológica:** El panorama de herramientas y marcos de big data cambia constantemente (por ejemplo, el paso de MapReduce a Spark, la aparición de nuevos motores de streaming). Esto obliga a las empresas a actualizarse continuamente, lo que puede generar obsolescencia de inversiones previas.
- **Dificultad para demostrar el valor (ROI):** Extraer *insights* valiosos no es automático; muchas iniciativas de big data fracasan porque no logran traducir los análisis en acciones concretas que impacten en el negocio, o porque los beneficios tardan en materializarse, provocando escepticismo entre los directivos.

1.2.3 ¿Cuándo utilizarlo?

El modelo *big data* se debe utilizar cuando las características de los datos (volumen, velocidad, variedad) o los requisitos de análisis superan las capacidades de las herramientas tradicionales (bases de datos relacionales, hojas de cálculo o almacenes de datos convencionales). A continuación, se detallan los escenarios concretos en los que este modelo resulta indispensable.

Casos de uso ideales

- **Datos masivos que no caben en sistemas tradicionales:** Cuando el volumen de información supera el terabyte o petabyte, y el almacenamiento o procesamiento en un único servidor resulta inviable (por coste, rendimiento o límites físicos). Es el caso de registros de transacciones bancarias, logs de aplicaciones web, datos de sensores industriales o archivos históricos de empresas.
- **Necesidad de procesamiento en tiempo real o streaming:** Cuando los datos fluyen continuamente y requieren análisis inmediato para tomar decisiones en fracciones de segundo. Ejemplos típicos son la detección de fraudes en tarjetas de crédito, la monitorización de redes sociales para gestionar crisis de reputación, o el control de tráfico en ciudades inteligentes.
- **Datos de naturaleza heterogénea (estructurados, semiestructurados y no estructurados):** Si la información proviene de fuentes muy diversas (JSON, XML, texto libre, imágenes, vídeo, audio) y no encaja fácilmente en el esquema fijo de una base de datos relacional. El modelo big data permite almacenar y procesar todos estos formatos en un mismo ecosistema sin necesidad de transformaciones previas rígidas.
- **Necesidad de integrar datos de múltiples fuentes externas e internas:** Cuando se desea combinar datos propios (ventas, inventario, CRM) con datos externos (clima, redes sociales, tendencias de mercado, datos demográficos) para obtener una visión más completa. Esta integración es compleja en sistemas tradicionales, pero nativa en plataformas de big data.
- **Análisis avanzado y modelado predictivo:** Cuando el objetivo es aplicar técnicas de minería de datos, aprendizaje automático o inteligencia artificial sobre conjuntos extensos para descubrir patrones ocultos, hacer predicciones o generar recomendaciones personalizadas. El big data proporciona la materia prima necesaria para entrenar modelos robustos.
- **Necesidad de escalabilidad elástica:** Cuando el volumen de datos crece de forma impredecible o estacional, y se requiere poder añadir o quitar capacidad de almacenamiento y cómputo sin interrupciones, aprovechando la nube y el escalado horizontal.

¿Cuándo NO es recomendable? El modelo big data no debe adoptarse en los siguientes casos:

- **Volúmenes de datos pequeños o moderados:** Si los datos caben cómodamente en una base de datos relacional y las consultas son rápidas con índices convencionales, la complejidad y el coste de una plataforma big data no se justifican.

- **Datos puramente estructurados y estables:** Cuando la información es homogénea, tiene un esquema fijo y no se prevé que crezca de forma explosiva, un almacén de datos tradicional (data warehouse) es más sencillo y eficiente.
- **Requisitos de latencia ultrabaja con datos muy simples:** Para operaciones transaccionales simples (ej. consulta de saldo, actualización de inventario) donde se necesita respuesta en milisegundos y los datos son pocos, una base de datos relacional o una caché en memoria es más adecuada.
- **Organizaciones sin madurez en datos:** Si no existe una cultura orientada a datos, ni personal capacitado (científicos, ingenieros de datos) ni procesos de gobernanza, implementar big data puede generar más problemas que beneficios, ya que los datos sin gobierno se convierten en un lastre.

1.2.4 Software que lo implementa

Existen múltiples sistemas que implementan bases de datos vectoriales, desde soluciones especializadas hasta extensiones sobre motores existentes y servicios en la nube. Entre los más destacados se encuentran:

- **Pinecone:** Base de datos vectorial completamente gestionada, orientada a entornos empresariales con alta disponibilidad y escalabilidad.
- **Milvus** (y Zilliz): Sistema de código abierto distribuido, optimizado para miles de millones de vectores, con soporte para GPU y múltiples índices (HNSW, IVF).
- **Qdrant:** Base de datos vectorial de código abierto en Rust, conocida por su alto rendimiento y su potente filtrado sobre metadatos.
- **Weaviate:** Plataforma de código abierto con API GraphQL, búsqueda híbrida y capacidad para generar embeddings de forma nativa.
- **Chroma:** Base de datos vectorial ligera y de código abierto, ideal para prototipado rápido y desarrollo local.
- **pgvector:** Extensión para PostgreSQL que añade el tipo de dato vectorial e índices HNSW, permitiendo búsqueda semántica sin salir del ecosistema Postgres.
- **Oracle AI Vector Search:** Capacidad nativa de Oracle Database 23ai que integra vectores como tipo de dato, índices y operadores SQL, facilitando consultas híbridas con datos transaccionales.
- **MongoDB Atlas Vector Search:** Funcionalidad vectorial integrada en MongoDB, que combina búsqueda semántica con filtros JSON y consultas de texto completo.
- **Elasticsearch y OpenSearch:** Motores de búsqueda distribuidos que soportan campos vectoriales y búsqueda k-NN, ideales para casos que requieren combinar relevancia semántica y filtrado avanzado.
- **Redis Stack:** Extensión de Redis que añade búsqueda vectorial con latencia ultrarrápida, adecuada para aplicaciones en tiempo real.
- **Servicios en la nube:** Todos los principales proveedores ofrecen soluciones gestionadas: Amazon OpenSearch Service, Azure AI Search, Google Cloud Vertex AI Vector Search y Databricks Vector Search, entre otros.

La elección del software depende del volumen de datos, los requisitos de filtrado, la infraestructura existente y el modelo de despliegue deseado. Para equipos que ya utilizan PostgreSQL, pgvector es la opción más natural; para proyectos de producción a gran escala, Milvus, Qdrant o los servicios en la nube ofrecen el mejor rendimiento y escalabilidad.